Zenvyra

Diagnostico de Modulos SaaS

O que ja temos | O que falta | Roadmap de evolucao

Junho / 2026

1. Visao Geral do Projeto

O Zenvyra e uma plataforma SaaS multi-tenant para gestao de catalogo e vendas, construida em Laravel 13 + Filament 5. Cada loja (tenant) e fisicamente isolada em seu proprio schema PostgreSQL, garantindo separacao total de dados entre clientes.

O sistema possui dois paineis distintos: o Painel Admin (operacao da plataforma) e o Painel do Lojista (operacao de cada loja). A seguir, e apresentado um diagnostico completo dos modulos existentes e dos que ainda precisam ser desenvolvidos para que o produto seja considerado um SaaS completo e comercializavel.

Stack tecnologica

Camada	Tecnologia	Versao
Backend	Laravel + PHP	13 / 8.3
Admin UI	Filament	v5
Frontend	Livewire + Tailwind CSS	v4 / v4
Banco de dados	PostgreSQL (schema por tenant)	---
Testes	PHPUnit	v12

2. Modulos Implementados (18 modulos)

Os modulos abaixo estao completamente implementados, com models, services, testes automatizados e interface Filament.

2.1 Multi-tenancy (Isolamento por Schema)

- Cada loja possui schema fisico isolado no PostgreSQL (sem coluna tenant_id).
- TenantSchemaManager: cria e provisiona o schema no momento de criacao da loja.
- TenantObserver: gera te_slug / te_schema automaticamente.
- Middleware SetTenantSearchPath: direciona o search_path por requisicao.
- Comando artisan tenant:migrate para manutencao de schemas.
- TenantIsolationTest: suite de testes valida que dados nao vazam entre lojas.

2.2 Autenticacao e Papeis

- Dois paineis Filament distintos: Admin (plataforma) e App (lojista).
- TenantPortalRole enum: owner e member.
- Policies por recurso: CategoryPolicy, CustomerPolicy, OrderPolicy, ProductPolicy, TeamMemberPolicy.
- TenantMemberOperatorScope: filtragem automatica por vinculo ativo.

2.3 Planos e Assinaturas

- Plan: nome, slug, preco, billing_period (monthly/yearly), trial_days, max_products, max_users, features (JSON).
- Subscription: ciclo completo de estados — trialing -> active -> past_due -> expired / canceled.
- SubscriptionManager: startEvaluation, startCommercialSubscription, activate, markPastDue, expire, cancel, refreshState.
- SubscriptionIntent: distingue lojas em Avaliacao das lojas Comerciais.
- Comando RefreshSubscriptionStates: atualiza estados expirados via scheduler.
- EnsureSubscriptionIsActive middleware: bloqueia acesso da loja se sem assinatura ativa.
- SubscriptionRenewalNoticeService: exibe banner de alerta N dias antes do vencimento.
- PlanLimitService: enforça max_products e max_users conforme plano contratado.
- config/subscriptions.php: grace_days, billing_cycle_days, evaluation_days — todos parametrizaveis via .env.

2.4 Gestao de Lojas (Admin)

- CRUD completo de Tenants com ativacao/desativacao.
- CRUD de Planos com toggle de status.
- CRUD de Assinaturas com selecao de plano e datas.
- CRUD de Administradores da plataforma.

- RelationManager de usuarios vinculados a loja.
- PurgeEvaluationDataAction: exclui dados de lojas de avaliacao expiradas.

2.5 Dashboard Admin

- PlatformOverviewWidget: total de lojas, assinaturas ativas, receita estimada.
- ExpiringEvaluationsWidget: lojas em avaliacao proximas do vencimento.
- SubscriptionStatusChart: distribuicao de assinaturas por status.
- StoresByPlanChart: lojas por plano.
- TenantsGrowthChart: crescimento de lojas ao longo do tempo.

2.6 Catalogo de Produtos

- Categorias: CRUD com politica de acesso por tenant.
- Produtos: simples (estoque direto) e com variacoes (estoque por variacao).
- ProductImage: multiplas imagens com flag is_primary; Observer limpa arquivo ao excluir.
- ProductVariation: SKU, preco e estoque individualizado.
- Toggle de status (ativo/inativo) direto na listagem.

2.7 Controle de Estoque

- StockManager: servico central para todas mutacoes de saldo.
- StockMovement: auditoria completa (entry, sale, refund, adjustment) com usuario, data, observacao.
- Estoque hibrido: produto simples vs. produto com variacoes (soma das variacoes).
- Alerta de estoque baixo: badge colorido na listagem e filtro dedicado.
- Bloqueio de venda: validacao de quantidade disponivel ao adicionar item ao pedido.
- TenantGestaoSettings.permitirEstoqueNegativo: configuracao por loja.

2.8 Pedidos

- Ciclo de vida completo: pending -> paid -> partially_refunded / refunded / canceled.
- OrderObserver: automacao de baixa de estoque (inventory_deducted_at evita dupla baixa).
- OrderLifecycleManager: cancelamento (apenas pedidos pending).
- OrderRefundManager: estorno parcial ou total por item, com devolucao opcional ao estoque.

2.9 Clientes

- CRUD de Clientes com campos obrigatorios configuráveis via TenantGestaoSettings.
- ViewCustomer: infolist com historico completo de pedidos do cliente.
- OrdersRelationManager: pedidos vinculados diretamente na tela do cliente.

2.10 Time / Membros da Loja

- TeamMember resource: convite, edicao e remocao de membros.
- TenantTeamService: centraliza logica de vinculo usuario <-> loja.
- TenantPortalRole: owner (dono da loja) e member (colaborador).

2.11 Comissoes de Vendedores

- CommissionCalculator: calcula comissao por item com base na taxa configurada.
- UserCommissionResolver: resolve a taxa de comissao do usuario logado.
- Modulo ativavel/desativavel por loja via TenantGestaoSettings.moduloVendedoresAtivo.
- SellersPerformanceChart: grafico de desempenho no dashboard do lojista.

2.12 Identidade Visual / Branding

- IdentidadeVisual: logo, cores primarias/secundarias, tipografia, dark mode.
- TenantTheme: tokens de tema aplicados ao painel do lojista.
- BrandingService: aplica a identidade visual dinamicamente na interface.
- IdentidadeVisualHistorico: auditoria de todas as alteracoes de branding.

2.13 Dashboard do Lojista

- DashboardOverviewWidget: total de pedidos, receita, clientes, produtos.
- MonthlyRevenueChart: receita mensal acumulada.
- PaymentMethodRevenueChart: receita por forma de pagamento.
- TopProductsChart: produtos mais vendidos.
- RecentOrdersWidget: ultimos pedidos em tempo real.
- StockMovementsWidget + StockProductsWidget: visao rapida de estoque.

2.14 Configuracoes da Loja

- TenantGestaoSettings: preferencias operacionais persistidas em JSON no tenant.
- Configuracoes: exige_telefone_cliente, exige_email_cliente, modulo_vendedores_ativo, permitir_estoque_negativo.

2.15 Retencao e Purge de Dados

- TenantOrderPurgeService: exclui dados de lojas de avaliacao apos expiracao do periodo de retencao.
- PurgeRetainedEvaluationData command: comando agendavel para limpeza automatica.
- te_data_retention_ends_at e te_data_purged_at: campos auditaveis no tenant.

2.16 Testes Automatizados

- 27+ arquivos de Feature Tests cobrindo todos os modulos.

- Cobertura: isolamento multi-tenant, ciclo de assinatura, estoque, pedidos, estornos, identidade visual, comissoes.
- PHPUnit v12 com factories para todos os modelos.
- Suite completa executavel com: php artisan test --compact.

3. O Que Falta para um SaaS Completo (11 modulos)

Os modulos abaixo sao essenciais para que o Zenvyra possa operar comercialmente de forma autonoma, sem intervencao manual do administrador na maioria dos fluxos.

3.1 Gateway de Pagamento [CRITICO]

- O proprio codigo comenta: "Fase 3 - webhook do gateway alimenta os mesmos metodos".
- Hoje toda cobranca e manual: o admin ativa/cancela assinaturas no painel.
- Necessario integrar Stripe, PagSeguro, Asaas, Pagar.me ou similar.
- Implementar webhooks de recebimento (payment.succeeded, payment.failed, subscription.renewed).
- Criar fluxo de checkout para o lojista assinar/renovar o plano.
- SubscriptionManager ja esta preparado para receber chamadas do gateway sem reescrever a logica.

3.2 Self-service / Onboarding Publico [CRITICO]

- Nao existe fluxo publico de cadastro. Lojas sao criadas 100% manualmente pelo admin.
- Necessario: landing page com CTA, formulario de cadastro, criacao automatica de tenant.
- Fluxo: cliente se cadastra -> tenant e provisionado -> assinatura inicia automaticamente.
- Integra com o gateway (item 3.1) para captura do metodo de pagamento no onboarding.

3.3 Portal de Faturamento do Lojista [ALTO]

- Nenhum modelo de Invoice existe no projeto.
- O lojista nao consegue ver historico de cobranças, faturas em aberto ou recibos.
- Implementar: Invoice model, listagem de faturas, download de PDF, link de segunda via.
- Integra com o webhook do gateway para criar Invoice a cada cobrança.

3.4 Upgrade / Downgrade de Plano (Self-service) [ALTO]

- Troca de plano so e possivel manualmente pelo admin master.
- Implementar pagina no painel do lojista: comparativo de planos, botao de upgrade/downgrade.
- Troca de plano deve recalcular proximo billing e ajustar limites imediatamente.

3.5 Notificacoes por E-mail [ALTO]

- Nenhuma Mailable ou Notification class existe no projeto.
- E-mails essenciais ausentes: boas-vindas, confirmacao de assinatura, aviso de vencimento, confirmacao de pedido ao cliente final.

- Implementar com Laravel Notifications + driver de e-mail (Mailgun, SES, Resend).
- Criar templates responsivos em Blade com identidade visual da plataforma.

3.6 Exportacao de Dados [MEDIO]

- SalesReportService existe mas sem saida para CSV, Excel ou PDF.
- Implementar exportacao de pedidos, clientes e movimentacoes de estoque.
- Sugestao: Laravel Excel (maatwebsite/excel) para XLSX e DomPDF para PDF.
- Exportacao de nota fiscal simplificada para o cliente final (PDF do pedido).

3.7 Vitrine Publica / Loja Virtual [MEDIO]

- Existem prototipos HTML (screen-catalogo.html, landing.html) mas sem rotas ou controllers funcionais.
- O sistema e 100% back-office — clientes finais nao conseguem navegar ou comprar.
- Implementar: catalogo publico por tenant (/{slug}/catalogo), carrinho, checkout.
- Integra com o gateway de pagamento (item 3.1) para processar pagamentos.

3.8 API REST / Integracoes Externas [MEDIO]

- Nenhuma rota em routes/api.php.
- Sem API, o Zenvyra nao consegue integrar com ERPs, marketplaces, apps mobile ou automacoes.
- Implementar API RESTful por tenant com autenticacao via Sanctum/API token.
- Endpoints prioritarios: produtos, pedidos, clientes, estoque.
- Webhook outbound: notificar sistemas externos de eventos (novo pedido, alteracao de status).

3.9 Recuperacao de Senha e Gestao de Conta [MEDIO]

- Fluxo de recuperacao de senha nao verificado no painel do lojista.
- Sem pagina de perfil do usuario no painel App (alterar senha, dados pessoais).
- Verificacao de e-mail no cadastro (email_verified_at ja existe no model User).

3.10 Log de Auditoria Geral [BAIXO]

- Estoque e estornos possuem trilha de auditoria, mas nao ha log geral de acoes do usuario.
- Implementar: quem criou/editou/excluiu cada registro e quando.
- Sugestao: spatie/laravel-activitylog.

3.11 Metricas e Analytics da Plataforma [BAIXO]

- O dashboard admin tem widgets basicos, mas sem MRR, Churn Rate, LTV.
- Implementar painel financeiro: MRR, ARR, churn, novas lojas x cancelamentos.
- Relatorio de uso por tenant: produtos cadastrados, pedidos/mes, usuarios ativos.

4. Resumo Executivo — Tabela de Status

Visao consolidada de todos os modulos com status atual e prioridade de implementacao.

Modulo	Status	Prioridade
Multi-tenancy por schema PostgreSQL	Implementado	---
Autenticacao e papeis	Implementado	---
Planos e assinaturas (ciclo completo)	Implementado	---
Gate de acesso por assinatura	Implementado	---
Limites de plano (produtos, usuarios)	Implementado	---
Gestao de lojas (Admin)	Implementado	---
Dashboard Admin	Implementado	---
Catalogo de produtos e variacoes	Implementado	---
Controle de estoque com auditoria	Implementado	---
Ciclo de pedidos (pending a refunded)	Implementado	---
Gestao de clientes	Implementado	---
Time / membros da loja	Implementado	---
Comissoes de vendedores	Implementado	---
Identidade visual por tenant	Implementado	---
Dashboard do lojista	Implementado	---
Configuracoes da loja	Implementado	---
Retencao e purge de dados	Implementado	---
Testes automatizados (27+ arquivos)	Implementado	---
Gateway de pagamento	**Ausente**	**CRITICO**
Self-service / onboarding publico	**Ausente**	**CRITICO**
Portal de faturamento (invoices)	**Ausente**	**ALTO**
Upgrade/downgrade de plano	**Ausente**	**ALTO**
Notificacoes por e-mail	**Ausente**	**ALTO**
Exportacao de dados (CSV/PDF)	**Ausente**	MEDIO
Vitrine publica / loja virtual	**Ausente**	MEDIO
API REST / integracoes externas	**Ausente**	MEDIO
Recuperacao de senha / perfil	**Parcial**	MEDIO
Log de auditoria geral	**Ausente**	BAIXO
Analytics e metricas da plataforma	**Parcial**	BAIXO

5. Roadmap Sugerido

Proposta de fases de desenvolvimento para transformar o Zenvyra num SaaS completamente autonomo e comercializavel.

Fase 3 — Monetizacao Autonoma (Prioridade Critica)

- Gateway de Pagamento: integracao com Stripe ou Asaas. Webhooks de cobranca automatica.
- Checkout & Self-service: pagina publica de cadastro + selecao de plano + captura de cartao.
- Portal de Faturamento: Invoice model, listagem e download de recibos PDF no painel do lojista.
- Notificacoes por E-mail: boas-vindas, confirmacao de pagamento, aviso de vencimento.
- Resultado: plataforma opera sem o admin precisar criar lojas ou cobranças manualmente.

Fase 4 — Crescimento e Retencao (Prioridade Alta)

- Upgrade/Downgrade Self-service: lojista troca de plano sem precisar de suporte.
- Exportacao de Dados: CSV/Excel para pedidos, clientes e estoque; PDF de pedido para o cliente final.
- Recuperacao de Senha: fluxo completo e pagina de perfil do usuario no painel App.
- Analytics da Plataforma: MRR, ARR, churn rate, LTV no dashboard admin.

Fase 5 — Expansao de Produto (Prioridade Futura)

- Vitrine Publica: catalogo navegavel por tenant, carrinho e checkout integrado ao gateway.
- API REST: endpoints autenticados por tenant para integracao com ERPs e marketplaces.
- Webhooks Outbound: notificar sistemas externos de eventos (novo pedido, alteracao de status).
- Log de Auditoria Geral: spatie/laravel-activitylog para rastreabilidade completa.
- Metricas Avancadas: relatorios de uso por tenant, comparativos de desempenho.

6. Conclusao

O Zenvyra possui uma base tecnica solida e bem testada. O core do produto — multi-tenancy, catalogo, estoque, pedidos, comissoes e identidade visual — esta completamente implementado com cobertura de testes adequada.

O que o separa de um SaaS completo e comercializavel sao os modulos de monetizacao autonoma: gateway de pagamento, onboarding self-service e notificacoes. Esses tres itens (Fase 3) sao o caminho critico para que a plataforma opere sem intervencao manual e possa escalar o numero de clientes.

As fases 4 e 5 agregam maturidade e diferencial competitivo, mas podem ser priorizadas apos o produto ja estar gerando receita.

18 modulos implementados | 11 modulos a desenvolver | 3 modulos criticos para Go-to-Market